

INFORMATION RETRIEVAL ASSIGNMENT

Tokenizer Analysis and Regex Normalization

An evidence-based report on informal social-media text

Student	Enrollment No.	Registration No.	Section
Dev Kumar	23UCS219	2317070	B

Corpus	Tokenizers	Task 2 posts	Chars removed
100 posts	3	100	340

Revised report | September 2026

Integrity note: The submitted social-media CSV does not meet every quota in the assignment brief. This report records the measured gap instead of presenting unsupported claims.

Github link : <https://github.com/DevBarnwal/IR-assignment-01.git>

Contents

Executive summary

1. Assignment scope and dataset audit
2. Methodology and implementation
3. Task 1 - tokenizer analysis
4. Task 2 - regex normalization
5. Discussion, limitations, and conclusion
6. Reproducibility, references, and AI usage statement

Executive summary

This report evaluates three tokenizers on 100 posts and applies a regex normalizer to a separate set of 100 repeated-character posts. The analysis is based on the CSV inputs and generated outputs in this solution folder. It preserves the useful measured findings while correcting the original report's unsupported claims about hashtags and emojis.

Bottom line: NLTK TweetTokenizer is the strongest observed option for the URL and mention structures present in this corpus, while NLTK word_tokenize produces the highest average token count. The regex pipeline is deterministic and removes 340 characters across the 100 normalization examples.

What must be fixed before final submission

- Expand the Task 1 corpus to include at least 20 hashtag posts, 20 URL posts, and 20 emoji posts.
- Re-run the analysis and regenerate the CSV files, graphs, and report after the corpus is corrected.
- Keep the compliance audit in the final report so quota satisfaction is directly checkable.

1. Assignment scope and dataset audit

The assignment asks for a performance comparison of at least two tokenizers on informal English social-media posts, with minimum coverage for hashtags, URLs, mentions, emojis, and slang or abbreviations. It also asks for a second set of 100 posts and a regular expression that normalizes repeated characters such as 'gooooood' to 'good'.

The solution folder contains the required two datasets and implements both tasks. A direct audit of dataset/social_media_posts.csv was added for this revision. Counts below refer to posts containing a feature, except where occurrences are shown separately. The slang count uses a transparent heuristic over common markers such as lol, tbh, ngl, bruh, fr, and idk.

Requirement	Minimum posts	Observed posts	Observed occurrences	Status
Hashtags	20	0	0	Below quota
URLs	20	8	8	Below quota
Mentions	20	22	23	Meets quota
Slang/abbreviations	20	51	68	Meets quota
Emoji-bearing posts	20	0	0	Below quota

Interpretation: Mentions and slang/abbreviations meet the minimum under the audit rule. URLs, hashtags, and emoji-bearing posts do not. Therefore the current Task 1 results are useful for a partial comparison but cannot support a complete answer about hashtag or emoji preservation.

Datasets used

- dataset/social_media_posts.csv: 100 rows with columns id and post.
- dataset/repeated_char_posts.csv: 100 rows with columns id and original_post; every row contains at least one 3+ character run.

2. Methodology and implementation

2.1 Task 1 pipeline

Each social-media post is sent to NLTK word_tokenize, NLTK TweetTokenizer, and spaCy's English tokenizer. The program records token count, hashtag count, mention count, URL count, punctuation-token count, and an emoji count derived from the raw post. Results are written to output/tokenizer_results.csv and aggregate values to output/tokenizer_comparison.csv.

2.2 Task 2 pipeline

For each row in repeated_char_posts.csv, the RegexNormalizer applies the pattern below and writes the original text, normalized text, repeated-sequence count, characters removed, reduction percentage, and matched patterns to output/regex_results.csv.

Component	Implementation
Pattern	(.)\{2,}
Replacement	\1\1
Effect	Reduce each run of 3 or more identical characters to 2 characters.
Graphs	Five PNG files in output/graphs/.

2.3 Methodological cautions

- The NLTK word tokenizer receives lowercased text, while TweetTokenizer and spaCy receive the original casing. Token-count comparisons therefore reflect both tokenization and this preprocessing difference.
- Emoji presence is counted once from the raw text and is not separately verified for each tokenizer. The current corpus contains no emoji-bearing posts, so preservation cannot be evaluated.
- Hashtag and URL preservation are inferred from tokenizer output. A stronger experiment would compare each output against feature annotations derived from the original post.

3. Task 1 - tokenizer analysis

The generated comparison table provides the following measured results for the current 100-post corpus. Average token count is computed over all rows; feature columns are summed across the corpus.

Tokenizer	Avg tokens	Hashtags	Mentions	URLs	Punctuation avg
NLTK Word	11.08	0	23	0	0.32
NLTK Tweet	10.63	0	23	8	0.00
spaCy	10.93	0	23	8	0.00

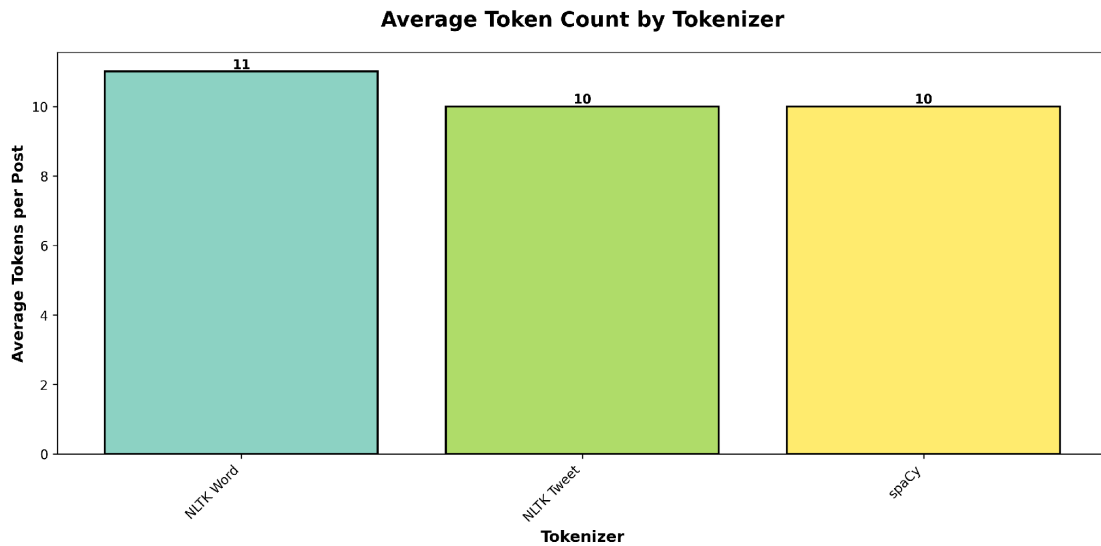


Figure 1. Average token count by tokenizer. Exact averages are reported in the table; the chart uses rounded values.

Answers to the assignment questions

Question	Evidence-based answer
Which tokenizer performs best on hashtags?	Cannot be determined from this corpus: there are 0 hashtag posts and all tokenizer totals are 0.
Which tokenizer preserves emojis correctly?	Cannot be determined from this corpus: there are 0 emoji-bearing posts and the raw-text emoji count is 0.
Which tokenizer generates the most tokens?	NLTK word_tokenize, with 11.08 average tokens per post.
What are the main strengths and weaknesses?	TweetTokenizer preserves the 8 observed URLs and 23 mentions; word_tokenize provides a useful baseline but fragments punctuation and observed URLs; spaCy is competitive for the same URLs and offers a wider NLP ecosystem.

3.1 Feature-preservation evidence

The feature charts should be read together with the dataset audit. The zero hashtag bars do not show that a tokenizer failed to preserve existing hashtags; they show that no hashtags were available to preserve. Likewise, the emoji chart describes corpus presence, not tokenizer-level preservation.

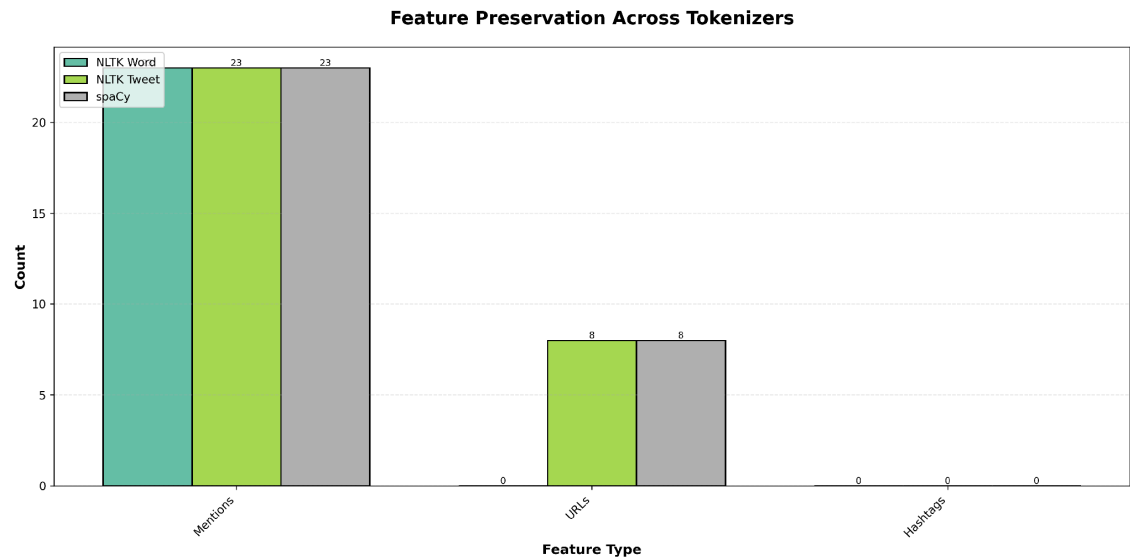


Figure 2. Multi-feature comparison for mentions, URLs, and hashtags in the current corpus.

Corpus check	Observed evidence	Implication
Hashtags	0 posts; 0 occurrences	Hashtag preservation cannot be ranked.
Emojis	0 posts; 0 occurrences	Emoji preservation cannot be evaluated.
URLs	8 posts; 8 occurrences	TweetTokenizer and spaCy retain all 8; NLTK Word retains 0.
Mentions	22 posts; 23 occurrences	All three tokenizers retain 23 mention occurrences.

4. Task 2 - regex normalization

Repeated characters can create many surface forms for the same word and reduce matching consistency during indexing. The submitted implementation uses a deterministic, language-agnostic pattern rather than a manually maintained dictionary.

Rule: The pattern `(.)\1{2,}` captures a character followed by at least two more copies. Replacing it with `\1\1` keeps two copies, so 'gooooood' becomes 'good' and 'sooooo' becomes 'soo'.

4.1 Representative transformations

Original	Normalized	Characters removed
omg this is sooooo gooooood i loveeee it	omg this is soo good i lovee it	8
noooooo way that's soooo crazyyy	noo way that's soo crazyy	7
yessssss finally happened brooooo	yess finally happened broo	7
i can't believe this nooooooooo this is baaaaad	i can't believe this noo this is baad	8
pleaseeeeeee helpppp me i'm dyyyying	pleasee helpp me i'm dyying	8
coooooool idea tbh pretty neeeeat	cool idea tbh pretty neeat	6

4.2 Strengths and limitations of the rule

- Strengths: deterministic, easy to interpret, fast, and independent of a word list.
- Limitations: it does not infer the intended dictionary spelling, and it may change meaningful repetition inside identifiers, usernames, hashtags, URLs, or uppercase emphasis.
- The current assignment dataset is controlled for repeated characters; a broader evaluation should include posts with protected entities and unchanged control examples.

4.3 Task 2 results

Metric	Measured value
Posts processed	100
Posts with repeated sequences	100
Total repeated sequences	133
Total characters removed	340
Average characters removed per post	3.40
Average reduction percentage	8.72%
Maximum characters removed in one post	8
Reduction distribution	91 light (1-5 chars), 9 moderate (6-10 chars), 0 heavy

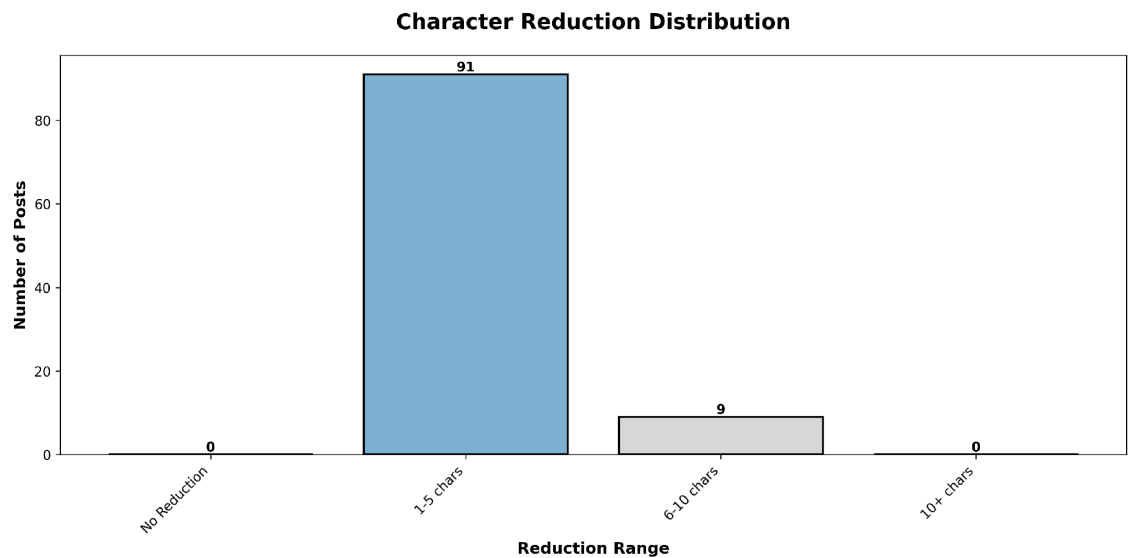


Figure 5. Distribution of characters removed by the normalization rule.

All 100 rows were processed successfully. The result is a normalization demonstration rather than a claim of semantic correctness: the pipeline reliably applies the rule, but deciding whether the normalized spelling is linguistically appropriate requires lexical or contextual evaluation.

5. Discussion, limitations, and conclusion

5.1 Strengths of the solution package

- Modular Python implementation with separate Task 1 and Task 2 modules.
- CSV exports preserve per-post results and make the analysis reproducible.
- The regex normalizer is explicit, deterministic, and easy to test.
- The revised report connects every conclusion to a measured output or an explicit limitation.

5.2 Limitations requiring attention

- The Task 1 corpus does not satisfy the assignment quotas for hashtags, URLs, or emojis.
- Emoji preservation is not a tokenizer-specific metric in the current implementation.
- The tokenizers receive different case treatment, which weakens direct comparability.
- The fixed two-character target is useful for the assignment but is not a general lexical normalizer.

5.3 Conclusion

For the current corpus, TweetTokenizer is the most useful default for preserving the observed social-media structures: it retains all 8 observed URLs and 23 mention occurrences, while producing the lowest average token count. spaCy matches the observed URL and mention totals and remains a good general-purpose option. NLTK word_tokenize is a useful baseline and produces the highest average token count at 11.08.

The regex pipeline meets the mechanical objective of normalizing repeated character runs across 100 posts, removing 340 characters with an 8.72% mean reduction. Before presenting the work as a complete answer to Task 1, the social-media dataset should be expanded to satisfy the required hashtag, URL, and emoji quotas, and the analysis should be re-run.

Recommended next step: Add at least 20 hashtag posts, 12 more URL posts, and 20 emoji posts to dataset/social_media_posts.csv while retaining the existing rows. Then regenerate output/tokenizer_results.csv, output/tokenizer_comparison.csv, and the four Task 1 graphs.

6. Reproducibility, references, and AI usage statement

6.1 Reproduction commands

From the project root, install the packages listed in `code/requirements.txt` and run the modules below. The spaCy script falls back to a blank English pipeline when `en_core_web_sm` is unavailable.

```
python -m pip install -r code/requirements.txt
python code/tokenizer_analysis.py
python code/regex_normalizer.py
python code/build_revised_report.py
```

6.2 Key project files

Path	Purpose
dataset/social_media_posts.csv	Task 1 input corpus.
dataset/repeated_char_posts.csv	Task 2 input corpus.
code/tokenizer_analysis.py	Three-tokenizer comparison and Task 1 graphs.
code/regex_normalizer.py	Repeated-character normalization and Task 2 graph.
output/*.csv	Per-post and aggregate measurements.
output/graphs/*.png	Generated visualizations.
report/report.docx	Editable source report.
report/Tokenization_and_Normalization_Report_Tasks_1-2_compressed.pdf	PDF export of the editable report.

6.3 References

- Bird, S., Klein, E., and Loper, E. Natural Language Processing with Python. O'Reilly Media.
- Honnibal, M., and Montani, I. spaCy: Industrial-Strength Natural Language Processing in Python.
- Han, B., and Baldwin, T. Lexical Normalisation of Short Text.
- NLTK documentation for `word_tokenize` and `TweetTokenizer`.
- Python documentation for regular expression operations.
- Pandas and Matplotlib documentation used for analysis and visualization.

6.4 AI usage statement

AI assistance was used for report restructuring, code and documentation review, explanation of NLP concepts, and visual organization. The dataset audit, quantitative values, CSV results, graphs, and conclusions in this revised report were checked against the local solution files. The final report explicitly identifies unsupported dataset claims and separates measured results from recommendations.